



09. Scheduling: Proportional Share

Until Now

Proportional Share Scheduler

Fair-Share Scheduler

Two scheduling algorithms: Lottery & Stride Scheduling

Lottery Scheduling

Basic Concept

Tickets

example

Ticket Mechanisms

Ticket currency (티켓 통화)

Ticket transfer (티켓 전송)

Ticket inflation

Implementation

U: unfairness metric

Lottery Scheduling의 Fairness는 어떠한가?

Lottery Scheduling Limitations

Short-term unfairness (단기적 불공평성)

Low predictability (낮은 예측 가능성)

Not real-time friendly (실시간 시스템에 부적합)

Overhead of randomness (랜덤 생성 오버헤드)

Scalability issues (확장성 문제)

Stride Scheduling

Stride

Scalability – Lottery vs. Stride Scheduling

Stride Scheduling (Fair Example)

Completely Fair Scheduling (CFS)

Concepts

Virtual Runtime (vruntime) → 누구 차례인지 결정

sched_latency (스케줄링 지연 시간)

min_granularity

왜 vruntime을 쓰는가

공평한 CPU 배분

결정적이고 비교 가능한 지표

시간이 지나면서의 공평성

Weight

Nice Value

CPU가 그냥 Nice Value 안 쓰고 Weight 사용하는 이유

time_slice 공식

예시

vruntime 공식

Structure of Ready Queue

Red-Black Tree

Dealing with I/O and Sleeping Processes

Problem

Solution

Challenge

Until Now

- 지금까지 FIFO, SJF, STCF, RR, MLFQ에서는 **Turnaround time, Response time** 이 주 고려사항
- 이제는 **fairness**가 주 고려사항

Proportional Share Scheduler

Fair-Share Scheduler

- CPU 시간을 "공정하게" 나눠주자!!
 - Proportional Share Scheduler는 각 작업(job)이 일정 비율의 CPU 시간을 확보 하도록 보장하도록 함으로서 수행
- turnaround나 response time 최적화가 목표는 아님
- **Fair-share scheduling**이 큰 개념이라고 보면 됨. **Proportional share scheduling**은 구체적인 구현 방법 중 하나

Two scheduling algorithms: Lottery & Stride Scheduling

- Lottery Scheduling: Allocates CPU **randomly** by ticket draw.

- Stride Scheduling: Allocates CPU **deterministically** using strides.

Lottery Scheduling

Basic Concept

- 무작위로 scheduler가 **winning ticket(당첨 티켓)** 을 선택
 - 당첨된 프로세스의 상태를 로드하여 실행

Tickets

- 프로세스가 받아야 할 리소스의 비율
- ticket의 비율: 해당 프로세스가 시스템 리소스에서 차지하는 비율을 의미

example

- there are two processes: A & B
 - Process A has 75 tickets → receive 75% of the CPU: 0 ~ 74
 - Process B has 25 tickets → receive 25% of the CPU: 75 ~ 99

Scheduler's winning tickets: 63 85 70 39 76 17 29 41 36 39 10 99 68 83 63

Resulting scheduler: A B A A B A A A A A A B A B A



시간이 길어질수록 = 경쟁이 오래 지속될수록, 단기적인 운의 편차(무작위성)가 사라지고 원래 설정한 비율(desired percentages)에 점점 가까워진다 - 큰 수의 법칙

Ticket Mechanisms

Ticket currency (티켓 통화)

- A user allocates tickets among their own jobs in whatever currency they would like.

- The system converts the currency into the correct global value
- **Example**

```
User A -> 500 (A's currency) to A1 -> 50 (global currency)
        -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)
```

- 총 200 tickets (Global currency)
- User A와 User B는 각각 100 tickets 가지고 있음(Global Currency)
- A는 2개의 프로세스를, B는 1개의 프로세스를 가지고 있음
 - A는 두 개의 프로세스에게 각각 500개의 A 티켓을 제공
 - B는 하나의 프로세스에 10개의 B 티켓을 제공
 - 이를 글로벌 티켓을 바꾸는 것
 - A의 두 프로세스는 50개의 글로벌 티켓, B의 프로세스는 100개의 글로벌 티켓

1단계: OS가 "어떤 user의 차례?" 결정

userA (100 티켓) vs userB (100 티켓) → lottery 돌려서 userA 당첨!

2단계: OS가 "userA의 어떤 process 실행?" 결정

process A1 (500 티켓) vs process A2 (500 티켓) → lottery 돌려서 A1 당첨!

Ticket transfer (티켓 전송)

- 프로세스는 일시적으로 다른 프로세스에게 자신의 티켓을 양도(hand off)
- 특히 클라이언트 / 서버 환경에서 유용

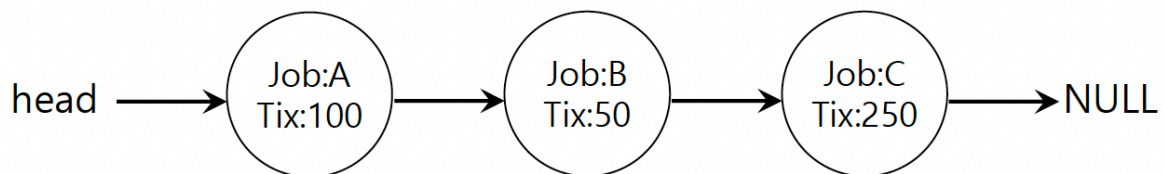
Ticket inflation

- 프로세스가 일시적으로 소유하는 티켓 수를 늘리거나 줄일 수 있음
- CPU time이 필요하면 추가적으로 티켓 발행 가능

```
User A -> 500 (A's currency) to A1 -> 60 (global currency)
        -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)
```

Implementation

- 3개의 프로세스 존재
 - 티켓 수는 총 400개
- Keep the processes in a list



```
// counter: used to track if we've found the winner yet
int counter = 0;

// winner: use some call to a random number generator to
// get a value, between 0 and the total # of tickets
int winner = getrandom(0, totaltickets);

// current: use this to walk through the list of jobs
node_t *current = head;

// loop until the sum of ticket values is > the winner
while (current) {
    counter = counter + current->tickets;
    if (counter > winner)
        break; // found the winner
    current = current->next;
}
// 'current' is the winner: schedule it...
```

U: unfairness metric

- The time the first job completes divided by the time that the second job completes



$U = (\text{첫 번째 job 완료 시간}) / (\text{두 번째 job 완료 시간})$

$U = 1.0$: 완벽하게 공평 (두 job이 동시에 끝남)

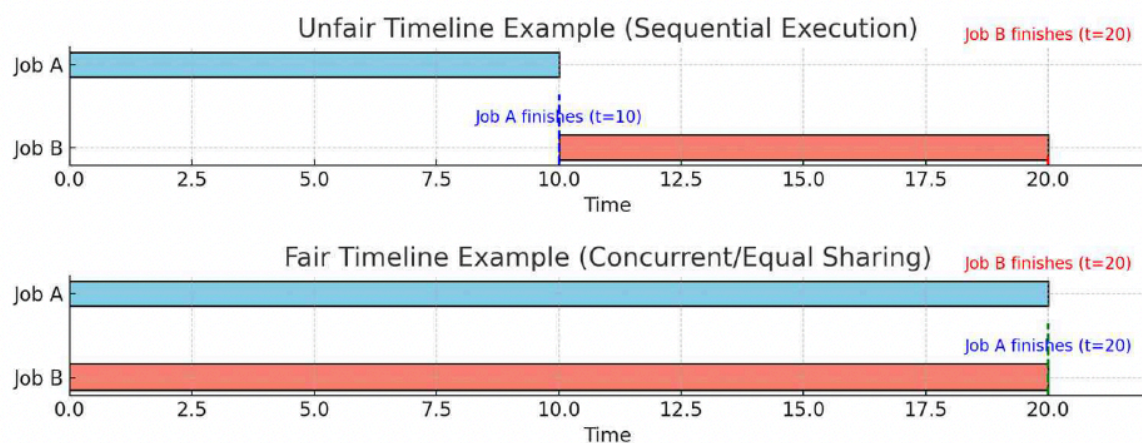
$U < 1.0$: 불공평함 (한 job이 훨씬 먼저 끝남)

U 가 0.5면: 한 job이 다른 job보다 2배 빨리 끝남

- Example:

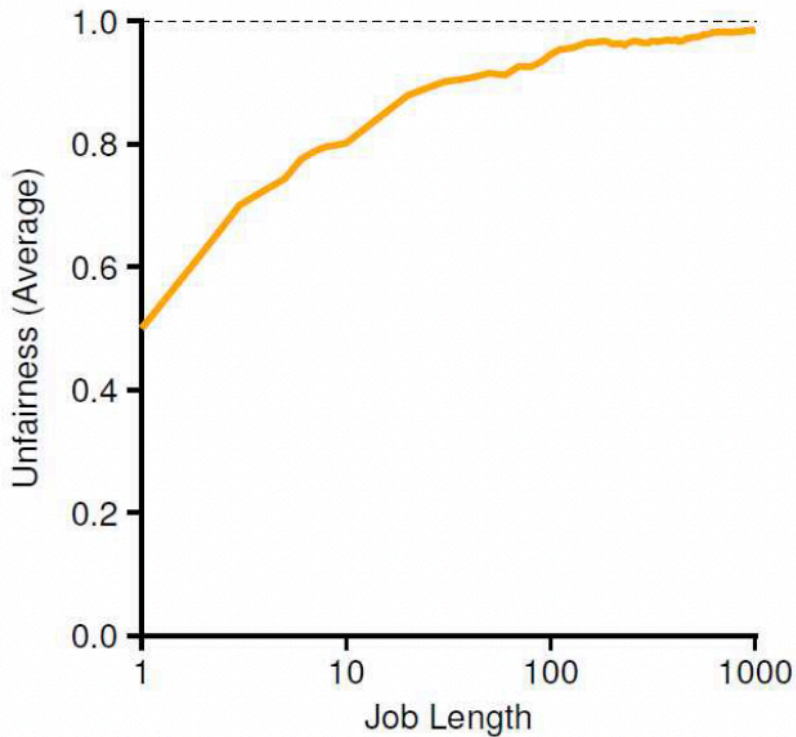
- There are two jobs, each jobs has runtime 10.
 - First job finishes at time 10
 - Second job finishes at time 20
 - $U = 10 / 20 = 0.5$
 - U will be close to 1 when both jobs finish at nearly the same time.

U가 1에 가까울수록 모든 job이 비슷한 시간에 완료돼서 공평한 거야!



Lottery Scheduling의 Fairness는 어떠한가?

- Each jobs has the same number of tickets (100)
각 Job은 100개의 ticket을 똑같이 가지고 있음



When the job length is not very long, average unfairness can be quite severe.

"Job이 짧으면 Lottery는 매우 불공평할 수 있다!"

- 동전 던지기 10번 하면 → 앞면 7번, 뒷면 3번 나올 수 있어 (불공평)
- 동전 던지기 1000번 하면 → 앞면 약 500번, 뒷면 약 500번 (공평)

Lottery Scheduling Limitations

Short-term unfairness (단기적 불공평성)

- Randomness may cause some jobs to dominate CPU in the short run.
랜덤이라서 짧은 시간 동안은 특정 job이 CPU를 독점할 수 있어

Low predictability (낮은 예측 가능성)

- Hard to guarantee precise completion times due to probabilistic nature.
확률적이라서 정확한 완료 시간을 보장하기 어려워

Not real-time friendly (실시간 시스템에 부적합)

- Cannot meet strict deadlines or real-time constraints.
엄격한 데드라인이나 실시간 제약을 만족시킬 수 없어

Overhead of randomness (랜덤 생성 오버헤드)

- Requires random number generation and ticket tracking at each scheduling decision.
스케줄링할 때마다 난수 생성하고 티켓 추적해야 해서 오버헤드 발생

Scalability issues (확장성 문제)

- Managing a large number of tickets can be inefficient.
티켓이 많거나 프로세스가 많으면 비효율적
- Lottery: requires traversing the process list until the cumulative sum exceeds the random "winner" $\rightarrow O(n)$.
프로세스 리스트를 처음부터 순회하면서 누적합 계산해야 해 - 프로세스 n 개 있으면 $\rightarrow O(n)$ 시간 복잡도

Stride Scheduling

- 10/13
- deterministic한 Proportional share 스케줄링 방식

Stride

- (A large number) / (the number of tickets of the process)
- 티켓이 많으면 보폭이 짧다?
- Example: a large number = 10,000
 - Process A has 100 tickets $\rightarrow$ stride of A is 100
 - Process B has 50 tickets $\rightarrow$ stride of B is 200
- A process runs, increment a counter(=pass value) for it by its stride.
프로세스가 실행되면, 그 프로세스의 카운터(=pass value)를 stride 값만큼 증가시킨다.

- Pick the process to run that has the lowest pass value
실행할 프로세스를 선택할 때는, pass value가 가장 낮은 프로세스를 고른다.

```
current = remove_min(queue); // pick client with minimum pass
schedule(current); // use resource for quantum
current→pass += current→stride; // compute next pass using stride
insert(queue, current); // put back into the queue
```

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

- C가 티켓을 가장 많이 가지고 있음
- 처음에 min값이 없어서 임의로 뽑음 - A
- B, C: 둘다 0이네 → 임의로 뽑았다고 가정?
- 근데 현재 시스템에서 새로운 process가 들어오면 문제점 0이면 계속 독점
pass-value값을 어떻게 줘야 하나: 현재 있는 값중 최소 값을 줌으로서 해결 가능

Advantage of Lottery scheduling: no per-process state

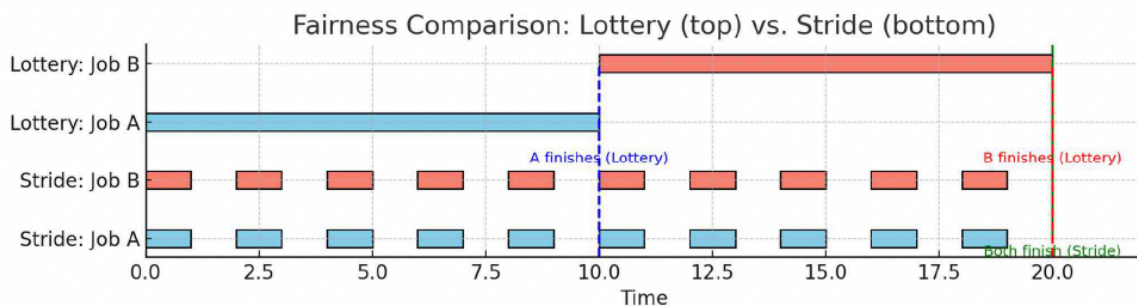
- 무슨 뜻이냐 → **Lottery**: 각 프로세스의 티켓 수만 알면 돼. Pass value 같은 누적 상태 필요 없어

- 그러나 Stride: 각 프로세스마다 pass value를 계속 추적하고 업데이트해야 해

Scalability – Lottery vs. Stride Scheduling

- Lottery Scheduling
 - Requires scanning tickets → $O(n)$ per decision
 - Inefficient with many processes or large ticket counts
- Stride Scheduling
 - Uses stride & pass values
 - With priority queue → $O(\log n)$ selection
 - More scalable and predictable

Stride Scheduling (Fair Example)



- **Stride가 Lottery보다 Fairness 측면에서 훨씬 낫다는 거야!**
- **In Lottery Scheduling, Job A completes fully before Job B starts, showing temporary unfairness due to randomness.**
 - Lottery는 랜덤이라 불공평할 수 있어
- **In Stride Scheduling, Jobs A and B alternate evenly and finish together, ensuring a fair and predictable outcome.**
 - Stride는 정확히 번갈아가며 공평하고 예측 가능해

Completely Fair Scheduling (CFS)

- 핵심목표: 모든 프로세스가 CPU 시간을 공정하게 나눠 가지도록 하는 것
- 리눅스 커널의 기본 CPU 스케줄러
- 고정되지 않은 타임슬라이스 (Non-fixed Timeslice)
 - 고정된 시간을 쓰지 않아
 - 가상 런타임(virtual runtime)으로 공정성(fairness) 추적
- 비례할당 (Proportional Allocation)
 - CPU 시간이 process weight에 비례하여 배분됨
 - 우선순위(Priority)는 nice value (-20 to +19) 값으로 조절 - 40 단계
 - nice값이 ↓ (-20에 가까울수록) 우선순위 ↑ → CPU 시간 ↑
 - nice값이 ↑ (+19에 가까울수록) 낮은 우선순위 ↓ → CPU 시간 ↓
- 효율적인 자료구조
 - Red-Black Tree로 task 관리
 - 탐색, 삽입, 삭제 → $O(\log n)$

Concepts

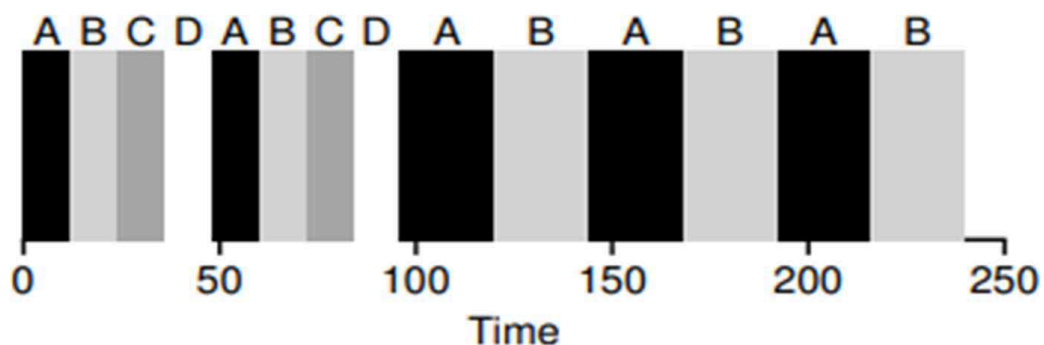
Virtual Runtime (vruntime) → 누구 차례인지 결정

- 프로세스별로 실행 시간을 추적하는 변수
 - 각 프로세스가 얼마나 많은 CPU 시간을 사용했는지 측정하는 내부 값
- 실제로 실행되는 동안 실시간에 비례하여 증가
- 증가 속도(growth rate)는 프로세스 우선순위(nice 값)에 의해 조정됨
 - 낮은 우선순위를 가진 프로세스(높은 nice 값)는 같은 시간 동안 CPU를 사용해도 vruntime이 더 빠르게 증가
 - 낮은 우선순위 프로세스에게 "너는 충분히 오래 실행된 것 같아"라는 페널티를 부여
 - 더 빨리 CPU에서 물러나도록

- CFS는 항상 가장 낮은 vruntime을 가진 프로세스를 다음에 실행할 프로세스로 선택
 - 왜? vruntime이 낮다는 것은 "가장 덜 실행된" 프로세스라는 것이기 때문에

sched_latency (스케줄링 지연 시간)

- 일반적인 값: 48 ms
- 모든 실행 가능한 프로세스가 최소 한 번은 CPU 시간을 할당 받아야 하는 기간
- **Process timeslice = sched_latency ÷ number of runnable processes**
- 프로세스 수가 증가하더라도 공정성을 보장



- 처음에는 A, B, C, D 프로세스 실행 / 어느 순간부터 A, B만 수행
- 근데 이런식으로 한다면 문제점: 만약 프로세스가 아주 많다면 어떻게 될까?
 - time slice가 너무 작아져서 context switch가 너무 자주 발생하게 될 것이고 이는 성능 저하로 연결
 - 이를 해결하기 위해 **min_granularity** 도입

min_granularity

- 최소 timeslice 보장 (**보통 6ms**)
- fairness는 나빠질 수 있지만 효율성 부분에서는 좋아짐

왜 vruntime을 쓰는가

공평한 CPU 배분

- vruntime 말고 실제 실행 시간(real runtime)만 추적하면 모든 프로세스를 똑같이 취급해서 우선순위를 무시하게 됨
 - vruntime은 **프로세스 가중치(nice 값에서 유도)**에 따라 실행 시간을 조정

- 높은 우선순위 → vruntime이 더 천천히 증가 → 더 자주 스케줄됨
- 낮은 우선순위 → vruntime이 더 빠르게 증가 → 덜 자주 스케줄됨

결정적이고 비교 가능한 지표

- CFS는 항상 vruntime이 가장 작은 프로세스를 선택
- 이걸 모든 프로세스에 걸쳐 하나의 비교 가능한 지표를 제공해서, 예측 가능한 스케줄링 결정을 보장

시간이 지나면서의 공정성

- 장기적으로 보면, 각 프로세스는 자신의 가중치(우선순위)에 비례하는 CPU 시간을 받음
- vruntime이 비례 배분 스케줄링(**proportional-share scheduling**)을 가능하게 만듦

Weight

Nice Value

```
static const int prio_to_weight[40] = {
    /* -20 */ 88761, 71755, 56483, 46273, 36291,
    /* -15 */ 29154, 23254, 18705, 14949, 11916,
    /* -10 */ 9548, 7620, 6100, 4904, 3906,
    /* -5 */ 3121, 2501, 1991, 1586, 1277,
    /* 0 */ 1024, 820, 655, 526, 423,
    /* 5 */ 335, 272, 215, 172, 137,
    /* 10 */ 110, 87, 70, 56, 45,
    /* 15 */ 36, 29, 23, 18, 15,
};
```

- -20부터 19까지 설정 가능
 - -20: 높은 우선순위 (Gets the most CPU time)
 - +19: 낮은 우선순위 (Gets the least CPU time)
- weight에 매핑됨

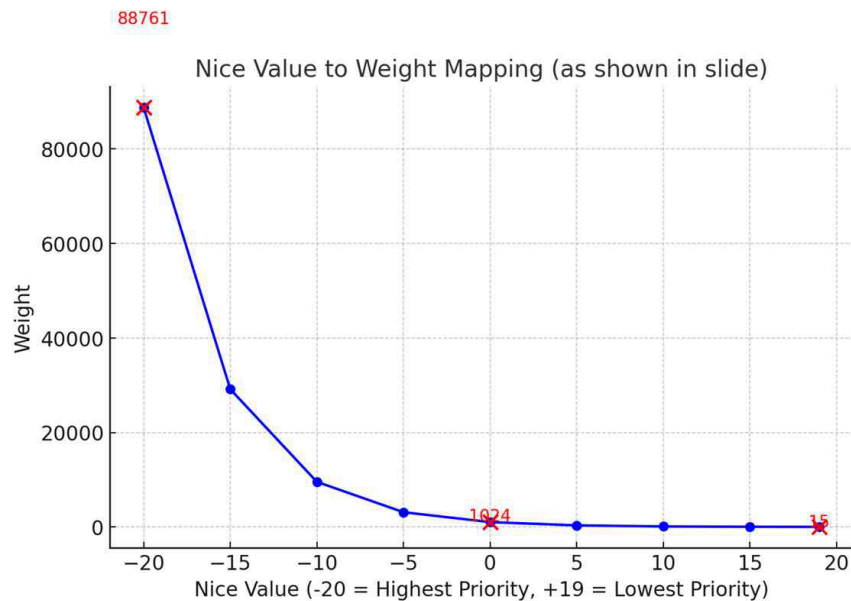
CPU가 그냥 Nice Value 안 쓰고 Weight 사용하는 이유

- Nice 값을 비례적인 CPU 배분으로 변환

- 만약 nice 값을 직접 쓰면
ex) A (nice=0), B (nice=5) → 비율이 0:5? 이상하잖슴~

- 비선형 매핑 (Non-linear Mapping)

- Weight가 지수적으로 증가해서 우선순위 차이가 더 명확해짐



- 공평한 vruntime 조정 (Fair vruntime Adjustment)

- 높은 weight → vruntime이 더 천천히 증가 → CPU 더 많이 받음

- 유연성 (Flexibility)

- 여러 프로세스간 fine-grained(세밀한) CPU 할당 제어 가능

time_slice 공식

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency}$$

예시

Process	nice value	weight	time slice
A	-5	3121	36ms

• A time slice 구하는 법
 $3121 / 4145 * 48 = 36.22926239$

B	0	1024	12ms
---	---	------	------

Weight(nice 값)는 ready queue에 있는 프로세스들끼리만 적용된다.

vruntime 공식

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

- $weight_0$ 은? nice = 0일때의 weight값
- $runtime_i$ 는 "실제로" 쓴 시간
 - 우선순위 낮으면? - nice값이 높음
 - weight값이 작지 → $weight_0 / weight_i$ 가 커서 accumulated value가 크게 됨
 - vruntime이 빨리 증가 (불리)
 - 우선순위 높으면? - nice값이 작음
 - weight값이 높음 → $weight_0 / weight_i$ 가 작아서 accumulated value가 작아짐
 - vruntime이 천천히 증가 (유리)

Process	nice value	weight	accumulated value
A	-5	3121	1 * runtime
B	0	1024	3 * runtime

Example: Why Larger Weight Slows Down vruntime Growth

▣ vruntime Growth

- ♦ vruntime increases as: $vruntime += \Delta t \times \frac{1024}{weight}$
 - ♦ For A: $vruntime_A += \Delta t$
 - ♦ For B: $vruntime_B += 0.5\Delta t$
- **B's vruntime grows more slowly**, so it gets picked more often.

▣ Time slice vs. Δt

- ♦ Time slice = the maximum execution time limit set by the scheduler.
- ♦ Δt = the actual execution time consumed, which can differ:
 - Between processes (A vs. B), and even for the same process across different runs.

Structure of Ready Queue

Red-Black Tree

- Balanced binary tree (can address worst-case insertion)
- Ordering of Red-Black Tree : $O(\log n)$
- Efficiently find the process with minimum virtual runtime.
- Only running (or runnable) processes are kept therein.

Dealing with I/O and Sleeping Processes

Problem

- A process may have a very small vruntime after sleeping, which could let it monopolize the CPU when it wakes up.

프로세스가 sleep한 후에 매우 작은 vruntime을 가질 수 있어서, 깨어났을 때 CPU를 독점할 수 있다.

Solution

- On wake-up, set the process's vruntime to the minimum value found in the redblack tree (ready queue)
깨어날 때, 프로세스의 vruntime을 red-black tree(ready queue)에서 찾은 최소값으로 설정한다.
- Prevents the process from starting with an unfair advantage.
이렇게 하면 프로세스가 불공평한 이점을 가지고 시작하는 것을 방지할 수 있다.

Challenge

- Conversely, processes that sleep frequently for short periods always rejoin the queue late. As a result, these processes may not receive enough CPU time and miss their fair share.
- Therefore, CFS must adjust to ensure that I/O-bound processes are not unfairly penalized.

반대로, 짧은 시간 동안 자주 sleep하는 프로세스들은 항상 늦게 큐에 다시 합류한다.

그 결과, 이런 프로세스들은 충분한 CPU 시간을 받지 못하고 공평한 몫을 놓칠 수 있다.

따라서 CFS는 I/O 중심 프로세스가 불공평하게 불이익을 받지 않도록 조정해야 한다.